HOW I THINK ABOUT BUILDING PRODUCTS

01 Start with the problem, not the solution
I spend more time understanding what's broken than deciding what to build.

02 Pace matters more than people think
I'd rather get something out by Friday and learn from it than plan for a month.

03 The best feature is often no feature
If I'm writing a help doc for it, something's wrong with the design.

04 Taste matters. Design matters more than we think.
I care about spacing, copy, and flow. Products that feel right get adopted.

05 Talk to users every week, not through dashboards
I learn more from one user call than a week of looking at charts.

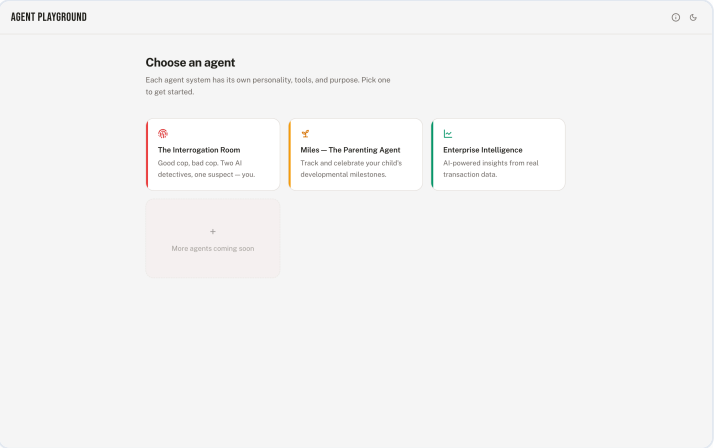
06 Own the last mile
I spend the extra hour on the edge case, the error state, the empty screen. That's what makes it complete.

MY EXPERIMENTS WITH AI

Agent Fleet

The infra layer I built from scratch. Front-end, back-end, everything needed to spin up a new agent without rebuilding the plumbing.

- Shared runtime handling SSE streaming, tool execution loops, and multi-agent handoffs
- Eval framework with Braintrust and LangFuse: automated datasets, LLM-as-judge scoring on accuracy, tone, and tool usage, regression testing before any prompt change ships
- Each agent gets its own system prompt, tools, and personality. The platform handles the rest.

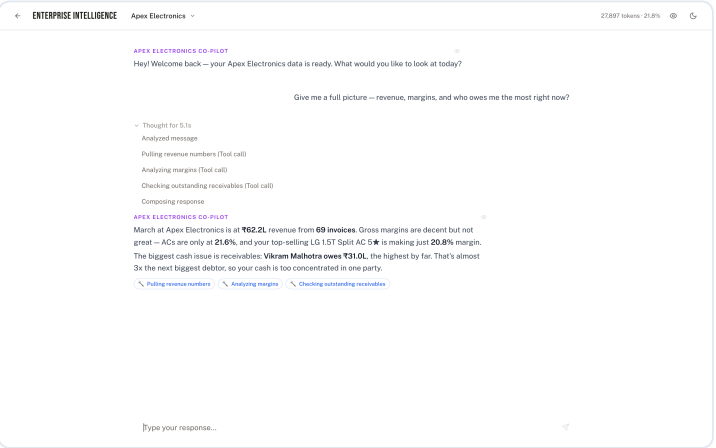


The platform. Each agent has its own tools and personality.

Enterprise Analytics Co-Pilot

An AI advisor that knows the business numbers and suggests what to do next. Not a dashboard, a conversation.

- **15+ custom tools** querying live data: revenue, margins, receivables, inventory, customer activity, price trends
- **RAG pipeline** (OpenAI embeddings + Pinecone) cross-referencing financials with business policies and enterprise knowledge
- **MCP integrations** pushing insights directly to Slack, Notion, and external tools without leaving the conversation
- Reasons across dimensions, fires **parallel tool calls**, and synthesizes one cross-functional answer
- **Eval suite** with 25+ test cases scoring on accuracy, tone, tool usage, and restraint. Braintrust + LangFuse integrated.

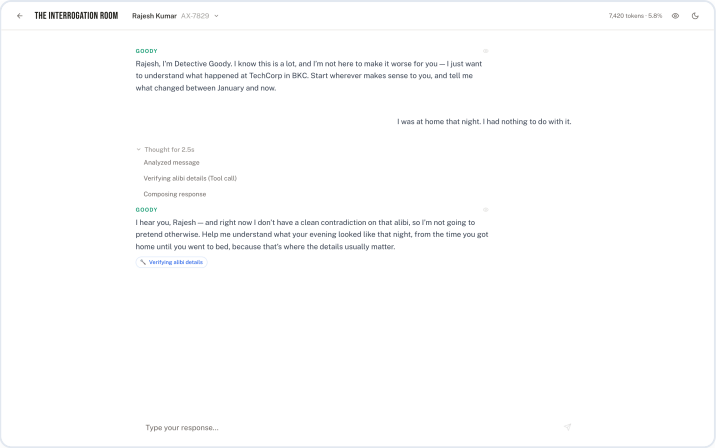


Parallel tool calls, one synthesized response with real numbers.

The Interrogation Room

Multi-agent system. Each cop has its own system prompt, personality, and tools.

- Good Cop builds rapport (empathy, open questions, deals). Bad Cop applies pressure (confrontation, urgency, victim impact). Different **system prompts** drive completely different behaviors.
- **12 tools with role-based access**: each agent can investigate, verify claims, apply pressure, or offer deals depending on their role
- **RAG-powered fact checking**: agents cross-reference suspect claims against case files in real time
- **Mid-conversation handoffs** with shared state: when one cop hits a wall, the other picks up with full context

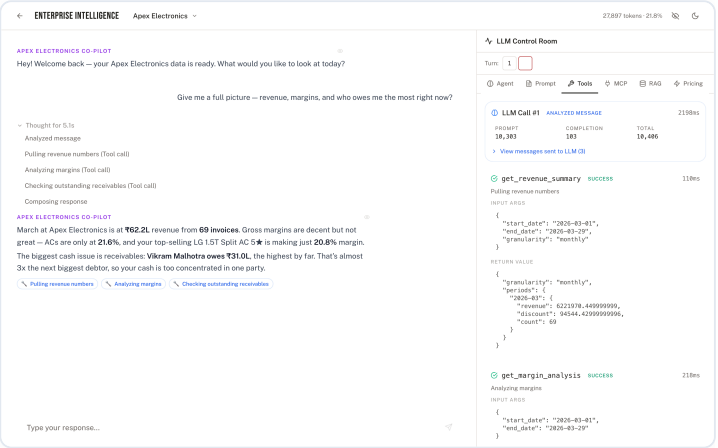


The agent thinks, verifies the alibi (tool call), then responds.

LLM Observability

Built into every agent. If I can't see why the agent did what it did, I can't improve it.

- **LangFuse integration** tracking every generation with session, system, and agent tags
- Full **tool call traces**: inputs, outputs, latency per call, all inspectable
- **Token economics**: system prompt, tool definitions, RAG context, conversation history, cost per turn
- RAG chunk inspection, MCP server status, and the complete prompt visible on every turn



Tool traces with full JSON payloads, latency, and token breakdown.